

YOUCODE · JAVA TRACK

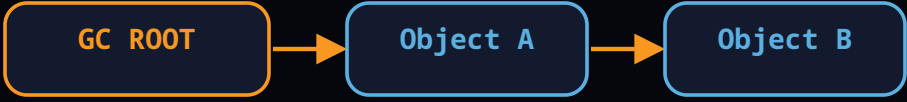
Java Memory
Management

GARBAGE COLLECTOR

Understanding how Java manages memory automatically — where objects live, how the Garbage Collector finds them, and why it matters when you write real code.

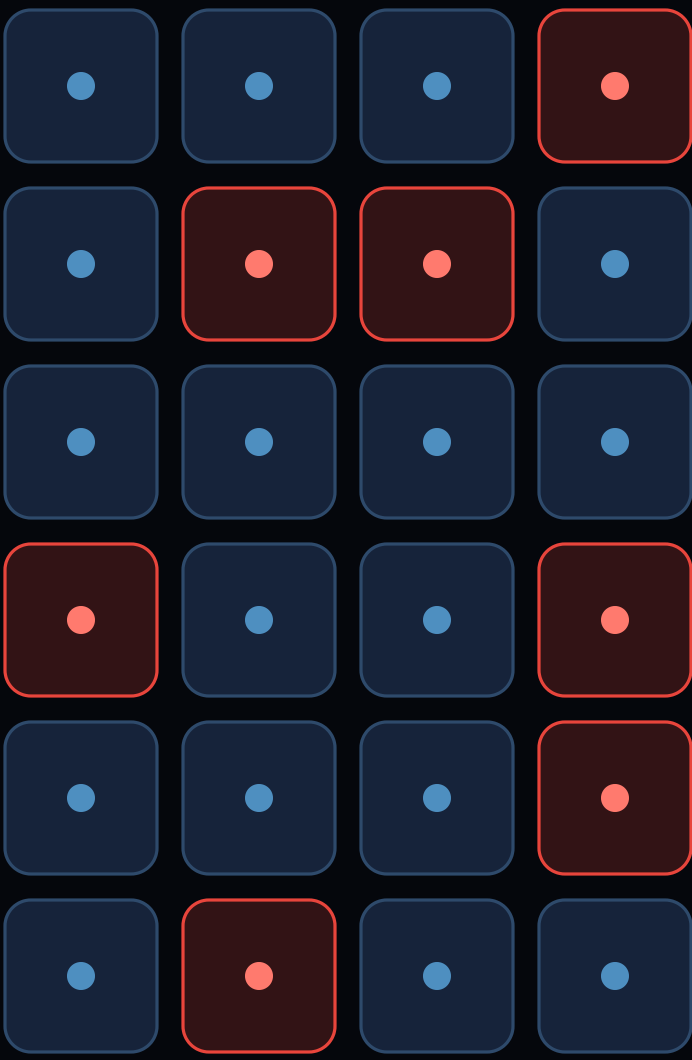
 Presenter · Kara Oussama  15 slides  15–25 min  JDK 17+ · G1 is the default

```
Object  
[Heap]  
  
Object obj = new Object(); // Allocation on the heap  
// If the reference is dropped  
obj is no longer in a container
```



no path from a root
→ unreachable

HEAP



 live object
reachable from a root

 garbage
unreachable → eligible

 reclaimed
memory available again



Mark · Sweep
Compact

a heap is memory the JVM manages for you

02 — The problem

Java Memory Management · Garbage Collector

Java Memory Management — Garbage Collector YouCode · Java Track 02 / 15

Why it matters at all

- Programs need memory to hold their **data** and their **objects**.
- Every new allocates memory that must eventually be **released**.
- In C / C++ you call `free()` yourself — forget it and you leak or crash.
- **Java** does it for you: the Garbage Collector reclaims memory automatically.

You never call `free()` in Java — but memory is still finite, so something has to clean up.
Objects are memory



Where does Java keep everything?

Two areas — the Stack for execution, the Heap for objects. Only the Heap is garbage-collected.

The 30-second model

- **STACK** — one per thread, per method call.
- Holds **locals**, parameters and **references** to objects.
- **HEAP** — shared memory for **objects** and arrays.
- Only the **Heap** is managed by the **Garbage Collector**.

Mental model: Stack = *what you are doing now* · Heap = *what you created*.

JVM memory map



Stack is for execution. Heap is for objects.

Variables live on the Stack. Objects live on the Heap — linked by references.

1 StackHeap.java Java

1 User user = new User();

2 int age = 20;

Read the two lines

- age holds a **value** directly.
- user holds a **reference** to a heap object.

One arrow explains half of Java memory

STACK · locals + refs

age = 20

user ● —┐

each frame disappears on return

reference



not a copy

HEAP · objects live here



“The stack remembers what you're doing. The heap stores what you created.”

User object

name · age · ...

05 — What is  reachable → GC keeps it

Java Memory Management · Garbage Collector

 Java Memory Management — Garbage Collector YouCode · Java Track 05 / 15

Stack = fast · per-call

Garbage = unreachable!

 Heap · shared · GC-managed

Nobody can reach the object any more — so it becomes eligible for collection.

1 GarbageDemo.java Java

1 User user = new User();

2

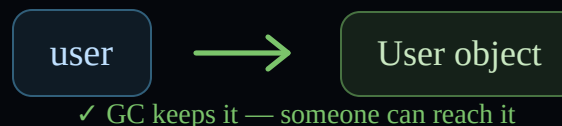
3 user = null; // reference dropped

An unreachable object becomes eligible for GC.

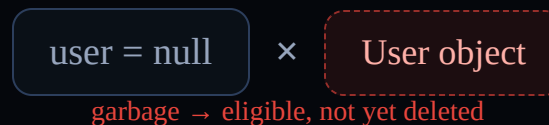
Drop the last reference → the object floats away

→

BEFORE · reachable



AFTER · unreachable



 **Eligible** ≠ deleted — the GC still has to run.

Setting `user = null` does NOT delete the object — it only cuts the last rope. The collector frees it later.

Same job — find garbage. Different trade-offs between throughput and pause time.



Serial GC

- Simple • **single-threaded**
- Small apps • tiny heaps



Parallel GC

- **Multi-threaded** stop-the-world
- Max **throughput** batch jobs



G1 GC ★

- **Region-based**, default JDK 9+
- Balances latency + throughput



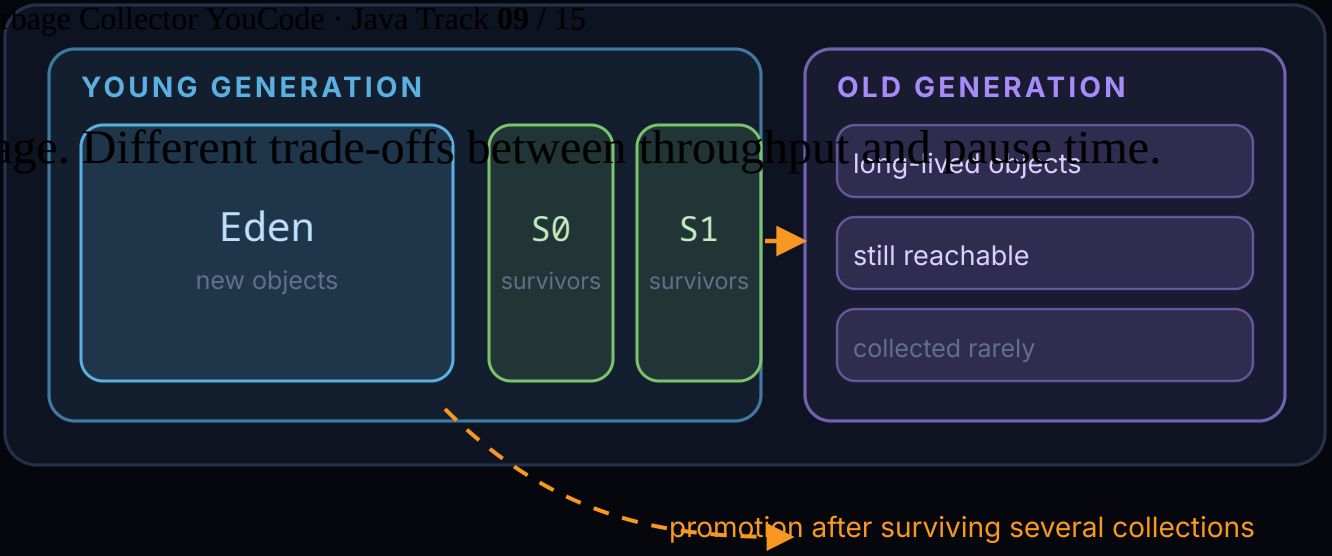
ZGC

- **Sub-millisecond** pauses
- Huge heaps • concurrent



Shenandoah

- **Low-pause**, concurrent



most objects die young — that is why the split exists

Rule of thumb: default to **G1** — reach for **ZGC** / **Shenandoah** only when pauses hurt. ★ = the one we zoom into next.

G1 cleans the dirtiest first.

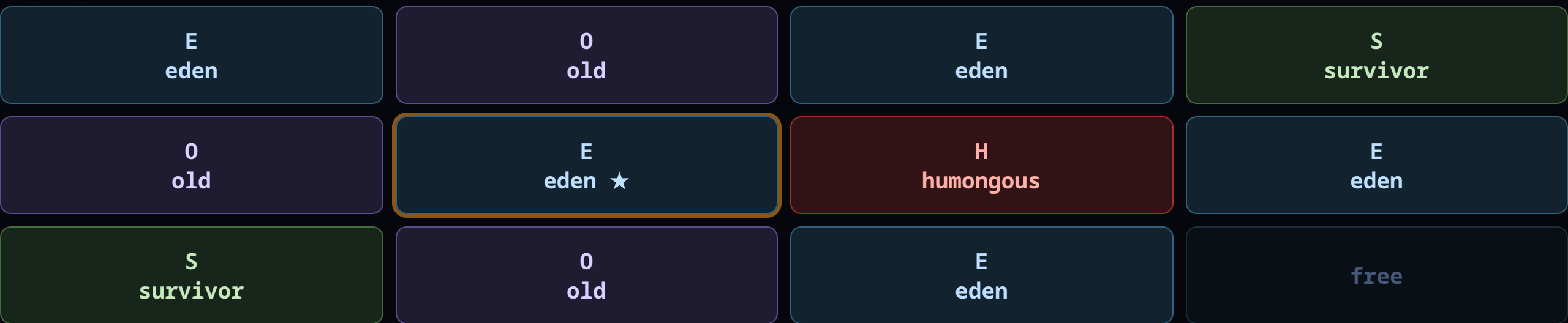
Garbage-First GC — the default since JDK 9. Regions, not generations-in-concrete.

G1 in 20 seconds

- G1 = **Garbage-First**. Heap split into equal **regions**.
- A region can be **Eden**, **Survivor**, **Old** or **Humongous**.
- G1 collects the regions with the **most garbage first**.
- Goal: big reclamation within a **pause-time target**.

E · Eden S · Survivor O · Old H · Humongous

Heap = regions · ★ collected first



★ = next collection target — most garbage per millisecond of pause

👉 “Garbage First: finally, a collector that knows what to clean first.”

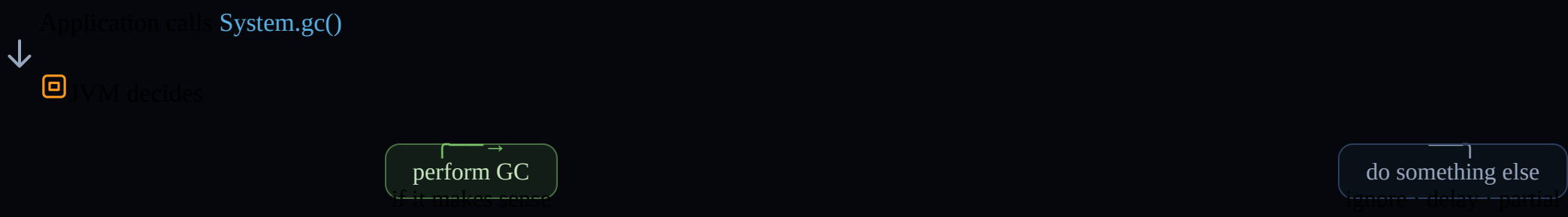
It suggests a collection — the JVM may obey, delay, or ignore it entirely.

```
1 Demo.java Java
2 public class Demo {
3     public static void main(String[] args) {
4         System.gc(); // hint, not a command
5     }
6 }
```

✗ Myth: “System.gc() forces GC.”

✓ Reality: it only hints the JVM to consider GC.

What really happens when you call it



does NOT guarantee anything — never rely on it for correctness

Birth → life → recycling.

Every Java object walks the same 6-step path — the GC owns the last two steps.

The whole story of an object, left to right



Created
new User()



Reachable
live refs exist



Used
read · write · call



Unreachable
refs dropped



Eligible
GC may take it





Reclaimed

memory reusable

ILifecycle.java Java

```
1 User user = new User("Oussama");
```

```
2
```

```
3 user = null; // unreachable → eligible
```

Map the code to the flow

`new User()` —→ Reachable + Used

`user = null` —→ Unreachable → Eligible

GC runs —→ Reclaimed ✓

Trace it: creation is yours, reclamation is the GC's. You only control the middle — the references.



+ still referenced
cache holds them



GC can't collect
reachable = kept



Heap grows
until it bursts



OutOfMemoryError
JVM gives up

14 — When memory runs out

Java Memory Management · Garbage Collector

Java Memory Management — Garbage Collector YouCode · Java Track 14 / 15

When memory runs out.

If reachable garbage grows faster than the GC can free it, the JVM fails fast: `OutOfMemoryError`.

1 DemoOOM.java Java

2 for (int i = 0; i < 1_000_000; i++) {

3 byte[] data = new byte[1024]; // 1 KB per loop

4 }

5

6 System.gc(); // still just a hint

kara@youcode — ~/gc-demo

limit the heap, then watch the GC

\$ java -Xms128m -Xmx256m App

run with a 128 MB → 256 MB heap

\$ java -Xlog:gc App

[0.023s][info][gc] GC(0) Pause Young ... 12M → 3M (48M)

the error every Java dev meets once

```
java.lang.OutOfMemoryError: Java heap space
```

- **Heap is finite** — `-Xmx` sets the ceiling.
- GC can only free the **unreachable**.
- Leaked / pinned objects pile up → **boom**.

`-Xms` · start heap · `-Xmx` · max heap · `-Xlog:gc` · GC logs

Your turn (60–90 seconds)

► **LIVE DEMO** — run it, shrink **-Xmx**, read **-Xlog:gc**, watch pauses + heap.

Tip: start at `-Xmx256m`, then try `64m` and compare the GC log.

Key takeaways.

Nine cards — if you remember these, you understand Java memory.

 Memory

Stack = calls · **Heap** = objects

 Objects

Allocated on the **Heap** via **new**

 Reachability

GC **Roots** → reachable = kept

 Garbage

Unreachable = eligible, not deleted

 Collector

Mark · **Sweep** · **Compact** the Heap

 Generations

Young (Eden+S) → **Old** via promotion

 Collectors

Serial · Parallel · **G1** ★ · ZGC · Shenandoah

 System.gc()

A **hint** — never a guarantee

 Leaks

Retained refs pin memory → OOM

Understanding memory helps Java developers write more efficient and reliable applications.

Questions?